

```

import { useState, useEffect } from "react";

const MATERIAL_FINGERPRINTS = {
  OBSIDIAN: {
    id: "OBSIDIAN",
    name: "Obsidian",
    color: "#1a1a2e",
    accent: "#e94560",
    desc: "Volcanic glass. Hardened by pressure. Cuts without warning.",
    signals: ["darkness", "power", "sharp wit", "edge aesthetics", "night", "urba
pioneer: "Raven Voss"
  },
  KINTSUGI: {
    id: "KINTSUGI",
    name: "Kintsugi",
    color: "#2d1b00",
    accent: "#f4a300",
    desc: "Broken and repaired with gold. Fracture is the feature.",
    signals: ["healing", "resilience", "transformation", "art", "restoration", "b
pioneer: "Calista Voss"
  },
  PEARL: {
    id: "PEARL",
    name: "Pearl",
    color: "#0d1b2a",
    accent: "#d4e8f0",
    desc: "Formed in silence. Luminous under pressure. Oceanic depth.",
    signals: ["grace", "mystery", "water", "softness", "iridescence", "quiet powe
pioneer: "Isolde Voss"
  },
  MYSTIC_PEARL: {
    id: "MYSTIC_PEARL",
    name: "Mystic Pearl",
    color: "#0a0a1a",
    accent: "#b8a9d9",
    desc: "Pearl meeting void. Spiritual gravity. Seen and unseen.",
    signals: ["spirituality", "duality", "philosophy", "cosmos", "authority", "wi
pioneer: "Marcus Kane"
  },
  FRACTURE: {
    id: "FRACTURE",
    name: "Fracture",
    color: "#1a0a0a",
    accent: "#ff6b35",

```

```

    desc: "Structural tension made visible. The fault line as identity.",
    signals: ["conflict", "tension", "opposing forces", "raw energy", "fire", "fr
pioneer: null
},
VOID: {
  id: "VOID",
  name: "Void",
  color: "#050508",
  accent: "#7fff7f",
  desc: "Negative space as presence. Absence that commands.",
  signals: ["minimalism", "silence", "absence", "negative space", "zen", "empti
pioneer: null
}
};

const BEHAVIORAL_ARCHETYPES = {
  SOVEREIGN: { id: "SOVEREIGN", name: "Sovereign", glyph: "𐌒", desc: "Commands th
SHADOW: { id: "SHADOW", name: "Shadow", glyph: "●", desc: "Operates in the unse
THRESHOLD: { id: "THRESHOLD", name: "Threshold", glyph: "△", desc: "Lives at tl
FORGE: { id: "FORGE", name: "Forge", glyph: "✕", desc: "Builds and breaks. Crea
WITNESS: { id: "WITNESS", name: "Witness", glyph: "◎", desc: "Sees everything.
SIGNAL: { id: "SIGNAL", name: "Signal", glyph: "𐌵", desc: "Pure transmission. T
ARCHITECT: { id: "ARCHITECT", name: "Architect", glyph: "O", desc: "Systems thi
FRICTION: { id: "FRICTION", name: "Friction", glyph: "≠", desc: "Necessary resi
};

const NARRATIVE_ROLES = {
  PROTAGONIST: { id: "PROTAGONIST", name: "Protagonist", tier: "PIONEER", desc: "
ANTAGONIST: { id: "ANTAGONIST", name: "Antagonist", tier: "PIONEER", desc: "Opp
FOIL: { id: "FOIL", name: "Foil", tier: "MID", desc: "Reveals protagonist throu
CATALYST: { id: "CATALYST", name: "Catalyst", tier: "MID", desc: "Triggers chan
COMMODITY: { id: "COMMODITY", name: "Commodity", tier: "BENCH", desc: "Archetyp
ECHO: { id: "ECHO", name: "Echo", tier: "BENCH", desc: "Amplifies existing forc
};

// The 28-class compound taxonomy
const TAXONOMY = [
  // PIONEER CLASS - Protagonist + high-signal fingerprints
  { id: "FGE-C001", name: "The Obsidian Sovereign", fp: "OBSIDIAN", ba: "SOVEREIG
  { id: "FGE-C002", name: "The Kintsugi Sovereign", fp: "KINTSUGI", ba: "SOVEREIG
  { id: "FGE-C003", name: "The Pearl Sovereign", fp: "PEARL", ba: "SOVEREIGN", nr
  { id: "FGE-C004", name: "The Mystic Sovereign", fp: "MYSTIC_PEARL", ba: "SOVERE

  // ANTAGONIST CLASS
  { id: "FGE-C005", name: "The Obsidian Friction", fp: "OBSIDIAN", ba: "FRICTION"
  { id: "FGE-C006", name: "The Void Architect", fp: "VOID", ba: "ARCHITECT", nr:

```

```

// MID-TIER - Foil + Catalyst
{ id: "FGE-C007", name: "The Kintsugi Threshold", fp: "KINTSUGI", ba: "THRESHOL"
{ id: "FGE-C008", name: "The Pearl Shadow", fp: "PEARL", ba: "SHADOW", nr: "FOI"
{ id: "FGE-C009", name: "The Obsidian Signal", fp: "OBSIDIAN", ba: "SIGNAL", nr
{ id: "FGE-C010", name: "The Fracture Forge", fp: "FRACTURE", ba: "FORGE", nr:
{ id: "FGE-C011", name: "The Mystic Witness", fp: "MYSTIC_PEARL", ba: "WITNESS"
{ id: "FGE-C012", name: "The Pearl Architect", fp: "PEARL", ba: "ARCHITECT", nr

// DEEP BENCH - Commodity + Echo (scalable archetype matrix)
{ id: "FGE-C013", name: "The Obsidian Forge", fp: "OBSIDIAN", ba: "FORGE", nr:
{ id: "FGE-C014", name: "The Kintsugi Signal", fp: "KINTSUGI", ba: "SIGNAL", nr
{ id: "FGE-C015", name: "The Pearl Echo", fp: "PEARL", ba: "WITNESS", nr: "ECHO"
{ id: "FGE-C016", name: "The Fracture Signal", fp: "FRACTURE", ba: "SIGNAL", nr
{ id: "FGE-C017", name: "The Void Shadow", fp: "VOID", ba: "SHADOW", nr: "COMMO"
{ id: "FGE-C018", name: "The Kintsugi Forge", fp: "KINTSUGI", ba: "FORGE", nr:
{ id: "FGE-C019", name: "The Mystic Threshold", fp: "MYSTIC_PEARL", ba: "THRESH"
{ id: "FGE-C020", name: "The Obsidian Witness", fp: "OBSIDIAN", ba: "WITNESS",
{ id: "FGE-C021", name: "The Pearl Threshold", fp: "PEARL", ba: "THRESHOLD", nr
{ id: "FGE-C022", name: "The Fracture Sovereign", fp: "FRACTURE", ba: "SOVEREIG"
{ id: "FGE-C023", name: "The Void Witness", fp: "VOID", ba: "WITNESS", nr: "ECH"
{ id: "FGE-C024", name: "The Kintsugi Architect", fp: "KINTSUGI", ba: "ARCHITEC"
{ id: "FGE-C025", name: "The Obsidian Threshold", fp: "OBSIDIAN", ba: "THRESHOL"
{ id: "FGE-C026", name: "The Pearl Signal", fp: "PEARL", ba: "SIGNAL", nr: "COM"
{ id: "FGE-C027", name: "The Mystic Forge", fp: "MYSTIC_PEARL", ba: "FORGE", nr
{ id: "FGE-C028", name: "The Fracture Echo", fp: "FRACTURE", ba: "FRICTION", nr
];

const TIER_COLORS = {
  PIONEER: { bg: "#1a0a00", border: "#f4a300", label: "#f4a300" },
  MID: { bg: "#0a001a", border: "#b8a9d9", label: "#b8a9d9" },
  BENCH: { bg: "#001a0a", border: "#4a9e6b", label: "#4a9e6b" }
};

export default function FGETaxonomy() {
  const [selected, setSelected] = useState(null);
  const [filterFP, setFilterFP] = useState("ALL");
  const [filterBA, setFilterBA] = useState("ALL");
  const [filterTier, setFilterTier] = useState("ALL");
  const [view, setView] = useState("GRID"); // GRID | DETAIL | PROMPT
  const [promptOutput, setPromptOutput] = useState("");

  const filtered = TAXONOMY.filter(c =>
    (filterFP === "ALL" || c.fp === filterFP) &&
    (filterBA === "ALL" || c.ba === filterBA) &&
    (filterTier === "ALL" || c.tier === filterTier)
  );
};

```

```

    const generatePrompt = (cls) => {
      const fp = MATERIAL_FINGERPRINTS[cls.fp];
      const ba = BEHAVIORAL_ARCHETYPES[cls.ba];
      const nr = NARRATIVE_ROLES[cls.nr];
      return `=== FGE CHARACTER BATCH ENGINE INPUT ===
CLASS ID: ${cls.id}
CLASS NAME: ${cls.name}
TIER: ${cls.tier}

- MATERIAL FINGERPRINT -
Fingerprint: ${fp.name}
Doctrine: ${fp.desc}
Visual signals: ${fp.signals.join(", ")}
${fp.pioneer ? `Pioneer anchor: ${fp.pioneer}` : "Pioneer anchor: UNASSIGNED (new

- BEHAVIORAL ARCHETYPE -
Archetype: ${ba.name} ${ba.glyph}
Core logic: ${ba.desc}
Behavioral traits: ${ba.traits.join(", ")}

- NARRATIVE ROLE -
Role: ${nr.name}
Tier: ${nr.tier}
Function: ${nr.desc}

- COMPOUND CLASS IDENTITY -
${cls.summary}

- NEMOTRON DEMOGRAPHIC SIGNALS -
Likely source records: ${cls.nemotron_signals.join(" | ")}

- GENERATION INSTRUCTION -
Build a character that embodies ${cls.name} according to FGE Canon doctrine.
Material fingerprint governs visual identity and aesthetic ground.
Behavioral archetype governs decision logic and power expression.
Narrative role governs story function and relational position.
All three axes must be legible simultaneously in the character's dossier.
Do not flatten any axis. Compound identity is the moat.
===`;
    };

    const handleSelect = (cls) => {
      setSelected(cls);
      setPromptOutput(generatePrompt(cls));
      setView("DETAIL");
    };

```

```

const fp_data = selected ? MATERIAL_FINGERPRINTS[selected.fp] : null;
const ba_data = selected ? BEHAVIORAL_ARCHETYPES[selected.ba] : null;
const nr_data = selected ? NARRATIVE_ROLES[selected.nr] : null;
const tier_data = selected ? TIER_COLORS[selected.tier] : null;

return (
  <div style={{
    minHeight: "100vh",
    background: "#06060a",
    fontFamily: "'Courier New', monospace",
    color: "#c8c8d0",
    padding: "0"
  }}>
    { /* Header */ }
    <div style={{
      borderBottom: "1px solid #2a2a3a",
      padding: "24px 32px 20px",
      display: "flex",
      justifyContent: "space-between",
      alignItems: "flex-end",
      background: "linear-gradient(180deg, #0c0c14 0%, #06060a 100%)"
    }}>
      <div>
        <div style={{ fontSize: "10px", letterSpacing: "4px", color: "#555566",
          FERAL GLOSS EMPIRE — CANON SYSTEM
        }}>
        </div>
        <div style={{ fontSize: "22px", letterSpacing: "2px", color: "#e8e8f0",
          ARCHETYPE TAXONOMY ENGINE
        }}>
        </div>
        <div style={{ fontSize: "11px", color: "#666677", marginTop: "4px", let
          {TAXONOMY.length} CLASSES · COMPOUND AXIS · NEMOTRON-SG GROUNDED
        }}>
        </div>
      </div>
      <div style={{ display: "flex", gap: "8px" }}>
        {["GRID", "DETAIL", "PROMPT"].map(v => (
          <button key={v} onClick={() => setView(v)} style={{
            padding: "6px 16px",
            background: view === v ? "#1e1e2e" : "transparent",
            border: `1px solid ${view === v ? "#6666aa" : "#2a2a3a"}`,
            color: view === v ? "#9999cc" : "#555566",
            fontSize: "10px",
            letterSpacing: "2px",
            cursor: "pointer"
          }}>{v}</button>
        ))}
      </div>
    </div>
  </div>

```

```

    { /* Filters */ }
    <div style={{
      padding: "16px 32px",
      borderBottom: "1px solid #161622",
      display: "flex",
      gap: "24px",
      alignItems: "center",
      flexWrap: "wrap"
    }}>
      <div style={{ fontSize: "9px", letterSpacing: "3px", color: "#444455" }}>

        <div style={{ display: "flex", gap: "6px", alignItems: "center" }}>
          <span style={{ fontSize: "9px", color: "#444455", letterSpacing: "2px"
            [{"ALL", "PIONEER", "MID", "BENCH"].map(t => (
              <button key={t} onClick={() => setFilterTier(t)} style={{
                padding: "3px 10px",
                background: filterTier === t ? "#1a1a2a" : "transparent",
                border: `1px solid ${filterTier === t ? "#6666aa" : "#222233"}`,
                color: filterTier === t ? "#aaaacc" : "#444455",
                fontSize: "9px", letterSpacing: "1px", cursor: "pointer"
              }}>{t}</button>
            ))}
          </div>

          <div style={{ display: "flex", gap: "6px", alignItems: "center", flexWrap
            <span style={{ fontSize: "9px", color: "#444455", letterSpacing: "2px"
              [{"ALL", ...Object.keys(MATERIAL_FINGERPRINTS)].map(f => (
                <button key={f} onClick={() => setFilterFP(f)} style={{
                  padding: "3px 10px",
                  background: filterFP === f ? "#1a1a2a" : "transparent",
                  border: `1px solid ${filterFP === f ? (f === "ALL" ? "#6666aa" : MA
                    color: filterFP === f ? (f === "ALL" ? "#aaaacc" : MATERIAL_FINGERP
                      fontSize: "9px", letterSpacing: "1px", cursor: "pointer"
                    }}>{f === "ALL" ? "ALL" : MATERIAL_FINGERPRINTS[f].name.toUpperCase()
                  ))}
                </div>

              <div style={{ marginLeft: "auto", fontSize: "9px", color: "#444455", lett
                {filtered.length} / {TAXONOMY.length} CLASSES
              </div>
            </div>

          { /* Main Content */ }
          <div style={{ padding: "24px 32px" }}>

            { /* GRID VIEW */ }

```

```

{view === "GRID" && (
  <div style={{
    display: "grid",
    gridTemplateColumns: "repeat(auto-fill, minmax(280px, 1fr))",
    gap: "12px"
  }}>
    {filtered.map(cls => {
      const fp = MATERIAL_FINGERPRINTS[cls.fp];
      const ba = BEHAVIORAL_ARCHETYPES[cls.ba];
      const tc = TIER_COLORS[cls.tier];
      const isSelected = selected?.id === cls.id;
      return (
        <div key={cls.id} onClick={() => handleSelect(cls)} style={{
          background: isSelected ? tc.bg : "#0c0c14",
          border: `1px solid ${isSelected ? tc.border : "#1e1e2a"}`,
          padding: "16px",
          cursor: "pointer",
          transition: "all 0.15s ease",
          position: "relative",
          overflow: "hidden"
        }}>
          <div style={{
            position: "absolute", top: 0, left: 0, right: 0, height: "2px"
            background: fp.accent
          }} />
          <div style={{ display: "flex", justifyContent: "space-between",
            <div style={{ fontSize: "9px", color: "#444455", letterSpacing: "0.5px",
              <div style={{ display: "flex", gap: "6px", alignItems: "center",
                <span style={{ fontSize: "14px" }}>{ba.glyph}</span>
                <span style={{ fontSize: "9px", color: tc.label, letterSpacing: "0.5px",
                  </div>
              </div>
            </div>
            <div style={{ fontSize: "13px", color: "#e0e0ee", marginBottom: "10px",
              {cls.name}
            </div>
            <div style={{ fontSize: "10px", color: "#666677", lineHeight: "1.2",
              {cls.summary}
            </div>
            <div style={{ display: "flex", gap: "6px", flexWrap: "wrap" }}>
              <span style={{ fontSize: "8px", color: fp.accent, border: `1px solid ${fp.accent}`, padding: "2px 5px",
                {fp.name.toUpperCase()}
              </span>
              <span style={{ fontSize: "8px", color: "#8888aa", border: "1px solid #8888aa", padding: "2px 5px",
                {ba.name.toUpperCase()}
              </span>
              <span style={{ fontSize: "8px", color: "#666677", border: "1px solid #666677", padding: "2px 5px",

```

```

        {cls.nr}
      </span>
    </div>
  </div>
);
}}}
</div>
)}

{/* DETAIL VIEW */}
{view === "DETAIL" && selected && (
  <div style={{ maxWidth: "800px" }}>
    <div style={{ marginBottom: "20px" }}>
      <button onClick={() => setView("GRID")} style={{
        background: "transparent", border: "none", color: "#555566",
        fontSize: "10px", letterSpacing: "2px", cursor: "pointer", paddin
      }}>← BACK TO GRID</button>
    </div>

    {/* Class header */}
    <div style={{
      background: tier_data.bg,
      border: `1px solid ${tier_data.border}`,
      padding: "28px",
      marginBottom: "16px",
      borderLeft: `4px solid ${fp_data.accent}`
    }}>
      <div style={{ fontSize: "9px", color: "#555566", letterSpacing: "3p
      <div style={{ fontSize: "26px", color: "#f0f0f8", letterSpacing: "1
      <div style={{ fontSize: "13px", color: "#9999aa", lineHeight: "1.6"
    </div>

    {/* Three axis breakdown */}
    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", ga
      {/* Fingerprint */}
      <div style={{ background: "#0c0c14", border: `1px solid ${fp_data.a
        <div style={{ fontSize: "8px", color: "#444455", letterSpacing: "
        <div style={{ fontSize: "14px", color: fp_data.accent, marginBott
        <div style={{ fontSize: "10px", color: "#666677", lineHeight: "1.
          {fp_data.pioneer && (
            <div style={{ fontSize: "9px", color: "#444455", letterSpacing:
              PIONEER: <span style={{ color: fp_data.accent }}>{fp_data.pio
            </div>
          )}
        </div>

      </div>

      {/* Behavioral */}

```

```

<div style={{ background: "#0c0c14", border: "1px solid #2a2a3a", padding: "10px" }}>
  <div style={{ fontSize: "8px", color: "#444455", letterSpacing: "1px" }}>
    <div style={{ fontSize: "20px", marginBottom: "4px" }}>{ba_data.generation}
    <div style={{ fontSize: "14px", color: "#9999cc", marginBottom: "4px" }}>{ba_data.generation}
    <div style={{ fontSize: "10px", color: "#666677", lineHeight: "1.2" }}>
      <div style={{ display: "flex", flexWrap: "wrap", gap: "4px" }}>
        {ba_data.traits.map(t => (
          <span key={t} style={{ fontSize: "8px", color: "#555566", border: "1px solid #2a2a3a", padding: "2px 5px" }}>{t}</span>
        ))}
      </div>
    </div>
  </div>
</div>

{/* Narrative Role */}
<div style={{ background: "#0c0c14", border: `1px solid ${tier_data.borderColor}`, padding: "10px" }}>
  <div style={{ fontSize: "8px", color: "#444455", letterSpacing: "1px" }}>
    <div style={{ fontSize: "14px", color: tier_data.label, marginBottom: "4px" }}>{tier_data.label}
    <div style={{ fontSize: "10px", color: "#666677", lineHeight: "1.2" }}>
      <div style={{ fontSize: "9px", color: "#444455", letterSpacing: "1px" }}>
        TIER: <span style={{ color: tier_data.label }}>{nr_data.tier}</span>
      </div>
    </div>
  </div>
</div>

{/* Nemotron signals */}
<div style={{ background: "#0c0c14", border: "1px solid #1e1e2a", padding: "10px" }}>
  <div style={{ fontSize: "8px", color: "#444455", letterSpacing: "1px" }}>
    <div style={{ display: "flex", flexWrap: "wrap", gap: "8px" }}>
      {selected.nemotron_signals.map(s => (
        <span key={s} style={{ fontSize: "10px", color: "#7777aa", border: "1px solid #2a2a3a", padding: "2px 5px" }}>{s}</span>
      ))}
    </div>
  </div>
</div>

{/* Generate prompt button */}
<button onClick={() => setView("PROMPT")} style={{
  width: "100%",
  padding: "14px",
  background: "#0c0c20",
  border: `1px solid ${tier_data.borderColor}`,
  color: tier_data.label,
  fontSize: "11px",
  letterSpacing: "3px",
  cursor: "pointer"
}}>
  GENERATE BATCH ENGINE PROMPT →
</button>
</div>

```

```

    })

    { /* PROMPT VIEW */ }
    {view === "PROMPT" && (
      <div style={{ maxWidth: "800px" }}>
        <div style={{ marginBottom: "20px", display: "flex", gap: "16px" }}>
          <button onClick={() => setView("DETAIL")} style={{
            background: "transparent", border: "none", color: "#555566",
            fontSize: "10px", letterSpacing: "2px", cursor: "pointer", paddin
          }}>← BACK TO DETAIL</button>
          {selected && (
            <span style={{ fontSize: "10px", color: "#666677", letterSpacing:
              {selected ? selected.name : "— SELECT A CLASS FIRST —"}
            </span>
          )}
        </div>

        {!selected ? (
          <div style={{ color: "#444455", fontSize: "12px", letterSpacing: "2
            SELECT A CLASS FROM THE GRID FIRST
          </div>
        ) : (
          <>
            <div style={{
              background: "#080810",
              border: "1px solid #2a2a3a",
              padding: "24px",
              marginBottom: "12px"
            }}>
              <div style={{ fontSize: "8px", color: "#444455", letterSpacing:
                CHARACTER BATCH ENGINE INPUT — COPY TO PIPELINE
              </div>
              <pre style={{
                fontSize: "11px",
                color: "#9999bb",
                lineHeight: "1.7",
                whiteSpace: "pre-wrap",
                margin: 0,
                fontFamily: "'Courier New', monospace"
              }}>{promptOutput}</pre>
            </div>
            <button
              onClick={() => navigator.clipboard?.writeText(promptOutput)}
              style={{
                padding: "10px 24px",
                background: "transparent",
                border: "1px solid #2a2a3a",

```

```

        color: "#666677",
        fontSize: "10px",
        letterSpacing: "2px",
        cursor: "pointer"
    }}>
    COPY TO CLIPBOARD
</button>
</>

    })
</div>
})

{/* No selection state for DETAIL/PROMPT */}
{(view === "DETAIL" || view === "PROMPT") && !selected && (
    <div style={{ color: "#444455", fontSize: "12px", letterSpacing: "2px",
        SELECT A CLASS FROM THE GRID TO VIEW DETAIL
    </div>
    })
</div>

{/* Footer */}
<div style={{
    borderTop: "1px solid #161622",
    padding: "16px 32px",
    display: "flex",
    justifyContent: "space-between",
    fontSize: "9px",
    color: "#333344",
    letterSpacing: "2px",
    marginTop: "40px"
}}>
    <span>FGE-TAX-V1.0 · {TAXONOMY.length} CLASSES · 3-AXIS COMPOUND</span>
    <span>PIONEER: {TAXONOMY.filter(c => c.tier === "PIONEER").length} · MID:
</div>
</div>

);
}

```